

Flink Kubernetes 模块

核心流程与设计说明

基于 Apache Flink 1.15.4 Release 分支源码分析

flink-kubernetes 模块完整解读

模块路径:

flink-kubernetes/src/main/java/org/apache/flink/kubernetes/

共 90+ 个核心 Java 源文件 | 154 个 Java 文件(含测试)

目录

- 一、模块概览与架构设计
- 二、核心类设计说明
- 三、Session 集群部署流程
- 四、Application 集群部署流程
- 五、TM Pod 资源管理流程
- 六、装饰器模式与 Pod 构建
- 七、Session vs Application 模式对比
- 八、K8s Service 暴露策略
- 九、高可用 (HA) 机制
- 十、参数与配置体系
- 十一、Pod 模板支持
- 十二、关键设计模式

一、模块概览与架构设计

1.1 模块定位

flink-kubernetes 模块是 Apache Flink 与 Kubernetes 集成的核心桥梁。负责将 Flink 集群(Session/Application)部署到 K8s，通过 K8s API 管理 JobManager Deployment 和 TaskManager Pod 的完整生命周期，支持 K8s 原生的服务发现、配置管理和高可用。底层基于 Fabric8 Kubernetes Client 库实现。

1.2 模块包结构

包名	职责	核心类数
o.a.f.kubernetes	集群描述符、资源管理驱动	4
o.a.f.kubernetes.kubeclient	K8s客户端接口/实现、FlinkPod抽象	8+
o.a.f.kubernetes.kubeclient.factory	JM/TM Pod工厂	2
o.a.f.kubernetes.kubeclient.decorators	装饰器: Pod增强	15
o.a.f.kubernetes.kubeclient.parameters	JM/TM参数体系	4
o.a.f.kubernetes.kubeclient.resources	K8s资源封装	15
o.a.f.kubernetes.kubeclient.services	Service类型策略	5
o.a.f.kubernetes.configuration	K8s配置项定义	6
o.a.f.kubernetes.entrypoint	JM进程入口点	5
o.a.f.kubernetes.highavailability	基于ConfigMap的HA	12
o.a.f.kubernetes.cli	Session CLI	1
o.a.f.kubernetes.utils	常量和工具类	2

1.3 核心类层次结构



图 1-1: 核心类层次结构

1.4 整体架构概览

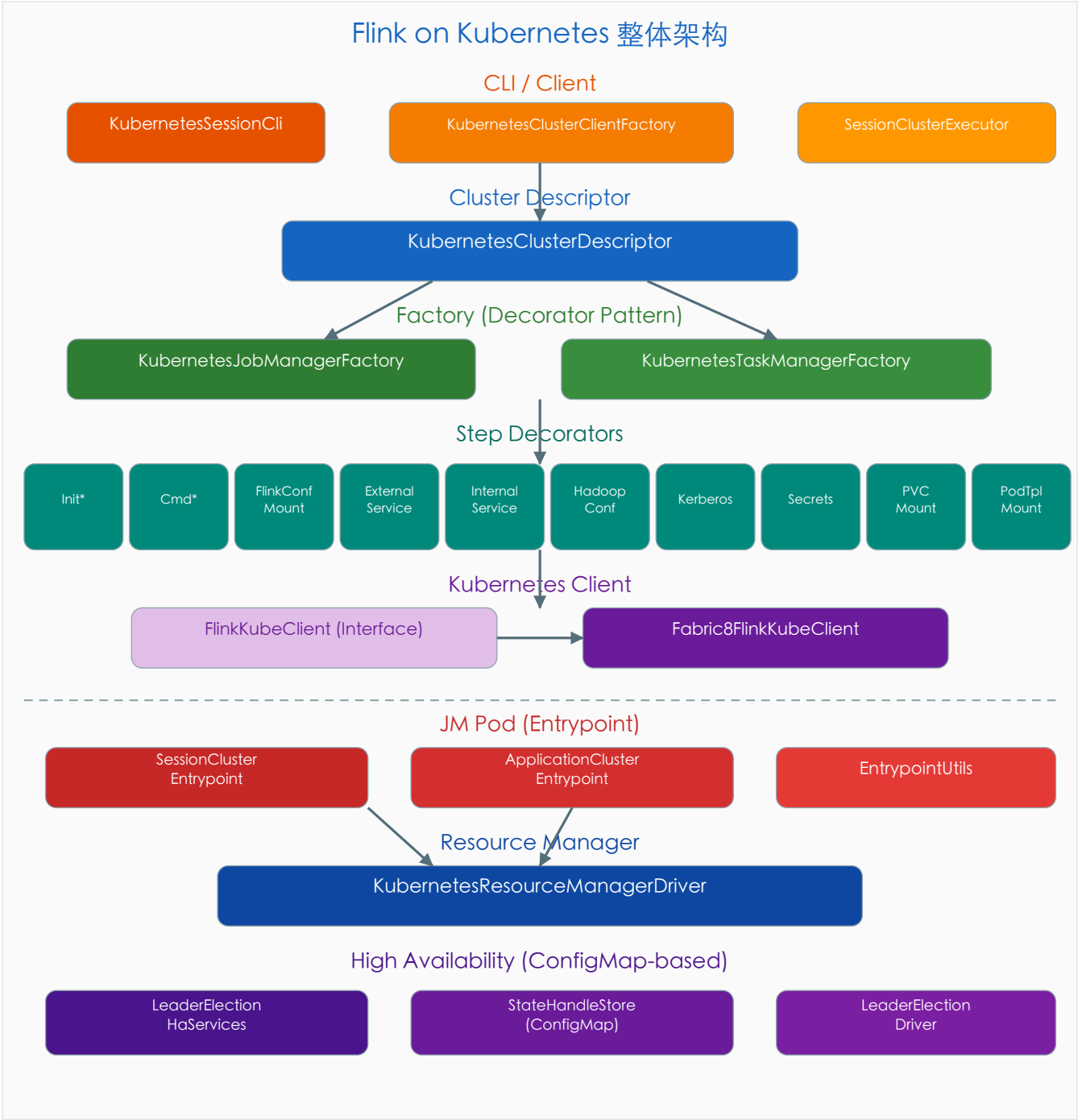


图 1-2: Flink on Kubernetes 整体架构

二、核心类设计说明

2.1 KubernetesClusterDescriptor (约331行)

实现 ClusterDescriptor<String> 接口(泛型参数为 clusterId 字符串)，是集群部署的核心协调者。

- deploySessionCluster(): 设置 entrypoint 为 KubernetesSessionClusterEntrypoint，调用 deployClusterInternal()
- deployApplicationCluster(): 设置 entrypoint 为 KubernetesApplicationClusterEntrypoint，可选上传用户 JAR (Artifact)
- deployClusterInternal(): 核心部署逻辑 — 构建 KubernetesJobManagerParameters -> 加载 Pod 模板 -> KubernetesJobManagerFactory 构建 JM Spec -> client.createJobManagerComponent()
- retrieve(clusterId): 根据 clusterId 获取已有集群的 RestClusterClient
- killCluster(clusterId): 清理集群全部 K8s 资源
- getClusterConnectionInfo(): 通过 REST Service 端点获取集群连接信息
- 核心成员: FlinkKubeClient client / FlinkKubeClientFactory clientFactory / KubernetesArtifactUploader artifactUploader

2.2 KubernetesResourceManagerDriver (约509行)

继承 AbstractResourceManagerDriver<KubernetesWorkerNode>，运行在 JM Pod 中，负责 TM Pod 的动态创建/销毁。

- initializeInternal(): 初始化 — Watch TM Pods + 加载 Pod 模板 + 恢复之前的 Worker 节点
- requestResource(): 创建 TM Pod: 构建 KubernetesTaskManagerParameters -> 装饰器链构建 Pod -> flinkKubeClient.createTaskManagerPod()
- releaseResource(): 停止 TM Pod (调用 flinkKubeClient.stopPod())
- recoverWorkerNodesFromPreviousAttempts(): RM Failover 后恢复已有的 TM Pod
- PodCallbackHandlerImpl 内部类: 处理 Pod Watch 事件 — onAdded(完成 Future) / onModified / onDeleted(回收资源) / onError
- Pod 命名规则: {clusterId}-taskmanager-{attemptId}-{podIndex}
- requestResourceFutures: 异步资源请求 Future 缓存，容器创建后 complete()
- currentMaxAttemptId / currentMaxPodId: Pod 命名计数器，确保唯一性

2.3 Fabric8FlinkKubeClient (约430行)

基于 Fabric8 Kubernetes Client 实现 FlinkKubeClient 接口，封装所有 K8s API 操作。

- createJobManagerComponent(): 创建 Deployment + 设置 OwnerReference + 创建附属资源(Service/ConfigMap)

- createTaskManagerPod(): 异步创建 TM Pod，设置 OwnerReference 关联到主 Deployment(级联删除)
- stopPod() / stopAndCleanupCluster(): 资源清理
- getRestEndpoint(): 获取 REST 服务端点(支持 ClusterIP/NodePort/LoadBalancer)
- watchPodsAndDoCallback(): Watch Pod 事件，带指数退避重试机制
- checkAndUpdateConfigMap(): 乐观锁事务更新 ConfigMap(用于 HA Leader 选举)
- loadPodFromTemplateFile(): 从 YAML 文件加载 Pod 模板
- kubeClientExecutorService: 专用异步 IO 线程池(默认4线程)
- masterDeploymentRef: 主 Deployment 引用(AtomicReference)，用于设置 OwnerReference

2.4 FlinkPod 抽象 (约96行)

核心 Pod 抽象，将 Pod 分为两个独立部分，使装饰器可以分别操作 Pod-level 和 Container-level 配置：

- podWithoutMainContainer: Pod spec (不含主容器)，用于设置 Volume、NodeSelector、Tolerations 等 Pod 级配置
- mainContainer: 主容器(名为 flink-main-container)，用于设置镜像、资源、端口、环境变量、VolumeMount 等
- Builder 模式: 提供链式构建 API，copy() 方法支持不可变拷贝
- 设计意义: 装饰器可独立修改 Pod 或 Container，避免修改冲突

2.5 KubernetesJobManagerFactory / TaskManagerFactory

静态工厂，通过装饰器链式调用构建 JM Deployment 和 TM Pod:

- KubernetesJobManagerFactory.buildKubernetesJobManagerSpecification(): 按顺序执行 11 个装饰器，将 FlinkPod 封装为 Deployment(含 replicas/selector/labels/ownerReferences)
- KubernetesTaskManagerFactory.buildTaskManagerKubernetesPod(): 装饰器链类似但不含 Service 相关装饰器，返回 KubernetesPod
- 装饰器执行顺序: Init -> EnvSecrets -> MountSecrets -> PVC -> Cmd -> InternalService -> ExternalService -> HadoopConf -> Kerberos -> FlinkConf -> PodTemplateMount
- 返回 KubernetesJobManagerSpecification: 封装 Deployment + 附属资源列表(Service/ConfigMap)

2.6 其他核心类

类名	职责
FlinkKubeClient	K8s 客户端接口: 定义 Pod/Service/ConfigMap 操作、Watch、Leader 选举等全部 K8s 交互 API

FlinkKubeClientFactory	单例工厂: 基于 Configuration 创建 Fabric8FlinkKubeClient, 配置命名空间/context/kubeconfig
KubernetesJobManager Specification	JM 规格封装: 持有 Deployment + 附属 K8s 资源列表(Service/ConfigMap)
KubernetesWorkerNode	Worker 节点抽象: 封装 ResourceID(格式: podName), 代表一个 TM Pod
KubernetesSessionCli	Session 集群 CLI: 解析 K8s Session 参数, 创建/连接 Session 集群, 使用 .yarn-properties 文件
KubernetesEntrypointUtils	入口工具类: 加载配置, HA 模式设置 Pod IP, HostNetwork 模式端口设为 0
KubernetesResourceManager Factory	RM 工厂(单例): 创建 KubernetesResourceManagerDriver, 内部创建 FlinkKubeClient
KubernetesUtils	工具类(637行): 构建启动命令、加载 Pod 模板、解析动态属性、构建 REST 端点
Endpoint	端点封装: 地址(address) + 端口(port), 用于 REST 服务
Constants	常量定义(138行): 容器名/卷名/挂载路径/端口名/标签键等全部 K8s 资源命名常量

三、Session 集群部署流程

Session 集群部署是 flink-kubernetes 模块最常用的部署模式。Session 集群会常驻运行一个 JM Deployment，后续通过 REST API 提交多个 Job 到同一集群。



图 3-1: Session 集群部署核心流程

关键实现细节:

- KubernetesClusterClientFactory 通过 SPI 机制自动发现，执行目标为 "kubernetes-session"
- deploySessionCluster() 设置 KubernetesConfigOptionsInternal.ENTRY_POINT_CLASS 为 KubernetesSessionClusterEntrypoint
- deployClusterInternal() 核心逻辑: 构建 JM Parameters -> 可选加载 Pod 模板 -> KubernetesJobManagerFactory 链式构建 -> createJobManagerComponent()
- createJobManagerComponent() 先创建 Deployment，然后设置 OwnerReference，再创建附属资源(Service/ConfigMap)
- OwnerReference 机制: 所有附属资源的 owner 设为 Deployment，删除 Deployment 时 K8s 自动级联删除
- 部署完成后轮询 REST 端点，等待 JM 启动并就绪后返回 RestClusterClient

四、Application 集群部署流程

Application 模式是生产推荐的部署方式。用户 JAR 在 JM Pod 内的 main() 方法中执行，Job Graph 在 AM 内生成，实现了 Job 级别的完全资源隔离。



图 4-1: Application 集群部署流程

与 Session 模式的关键差异:

- Artifact 上传: 通过 KubernetesArtifactUploader 将用户 JAR 上传到分布式存储(DFS)，或直接打入容器镜像

- 入口点类: `KubernetesApplicationClusterEntrypoint` 在 AM Pod 内执行用户 `main()` 方法
- `main()` 执行流程: 安装 `FileSystem` -> 安装 `SecurityContext` -> 获取 `PackagedProgram` -> 调用 `invokeInteractiveModeForExecution()`
- JM Pod 内部构建 `JobGraph`，无需客户端参与，减少客户端资源消耗
- 每个 Application 独立拥有 JM + TMs，Job 级别完全隔离

五、TM Pod 资源管理流程

资源管理流程运行在 JM Pod 内部，由 KubernetesResourceManagerDriver 驱动。通过 Fabric8 K8s Client 创建/删除 TM Pod，并通过 Pod Watch 机制感知 Pod 生命周期事件。



图 5-1: TM Pod 资源管理核心流程(时序图)

核心交互流程:

- 初始化: initializeInternal() 启动 Pod Watch、加载 TM Pod 模板、恢复之前的 Worker 节点
- 资源请求: SlotManager 通过 requestResource() 发起，驱动创建新的 TM Pod
- Pod 构建: KubernetesTaskManagerFactory 通过装饰器链构建完整的 TM Pod spec
- 异步创建: flinkKubeClient.createTaskManagerPod() 在 IO 线程池异步执行

- Pod Watch 回调: onAdded(完成 requestResource Future) / onModified(更新状态) / onDeleted(回收资源) / onError(错误处理)
- RM Failover 恢复: recoverWorkerNodesFromPreviousAttempts() 列出现有 TM Pod 并恢复为 Worker 节点
- 资源释放: releaseResource() 调用 flinkKubeClient.stopPod() 删除 Pod
- OwnerReference: TM Pod 的 owner 设为 JM Deployment , Deployment 删除时自动级联删除所有 TM Pod

六、装饰器模式与 Pod 构建

装饰器模式是 flink-kubernetes 模块最核心的设计模式。KubernetesStepDecorator 接口定义了两个方法：`decorateFlinkPod()` 增强 Pod 配置，`buildAccompanyingKubernetesResources()` 构建附属 K8s 资源。每个装饰器关注单一职责，通过链式调用组合出完整的 Pod 规格。

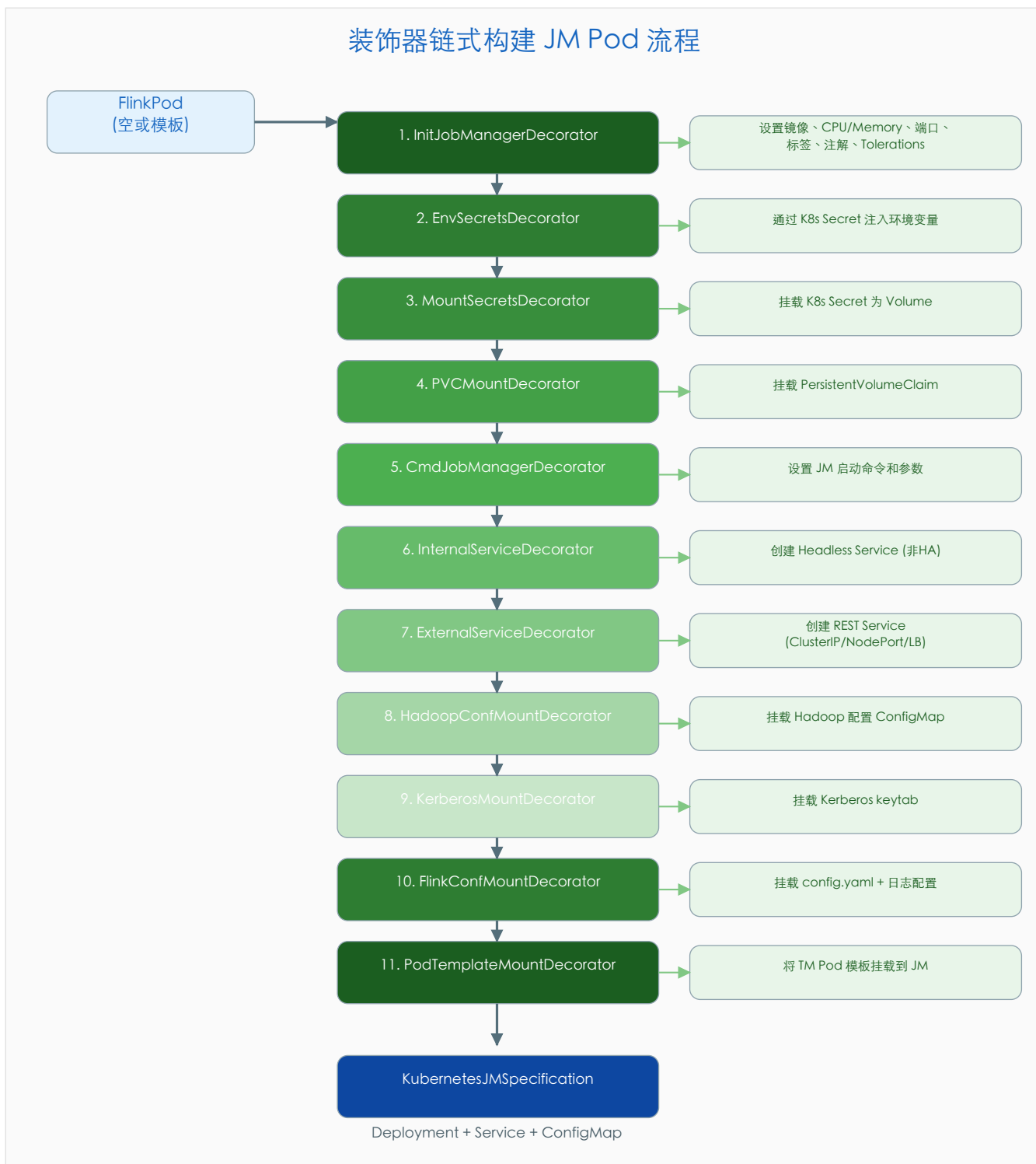


图 6-1: 装饰器链式构建 JM Pod 流程

各装饰器详细职责：

装饰器	操作对象	具体职责
InitJobManagerDecorator	Pod + Container	设置镜像、CPU Request/Limit、Memory、端口 (REST/RPC/Blob)、标签、注解、NodeSelector、Tolerations、ServiceAccount
InitTaskManagerDecorator	Pod + Container	类似 JM Init + 设置 blockedNodes 反亲和性
CmdJobManagerDecorator	Container	设置 JM 启动命令: kubernetes-jobmanager.sh + 参数
CmdTaskManagerDecorator	Container	设置 TM 启动命令 + JVM 内存参数 (-Xms/-Xmx/-XX:MaxDirectMemory/-XX:MaxMetaspace)
ExternalServiceDecorator	附属资源	创建 REST Service (类型由 kubernetes.rest-service.exposed.type 控制)
InternalServiceDecorator	附属资源	创建 Headless Service (非 HA 场景，用于 JM/TM 内部通信)
FlinkConfMountDecorator	Pod + Container + 附属资源	创建 ConfigMap(含 config.yaml + log4j/logback 配置)，挂载到 /opt/flink/conf
HadoopConfMountDecorator	Pod + Container	挂载已有的 Hadoop 配置 ConfigMap，设置 HADOOP_CONF_DIR 环境变量
KerberosMountDecorator	Pod + Container + 附属资源	挂载 Kerberos keytab 和 krb5.conf，设置 KRB5_CONFIG 环境变量
MountSecretsDecorator	Pod	将 K8s Secret 作为 Volume 挂载到指定路径
EnvSecretsDecorator	Container	通过 secretKeyRef 将 K8s Secret 值注入为环境变量
PodTemplateMountDecorator	Pod + Container + 附属资源	将 TM Pod 模板内容写入 ConfigMap，挂载到 JM Pod 的 /opt/flink/pod-template.yaml
PVCMountDecorator	Pod + Container	挂载 PersistentVolumeClaim，支持 readOnly 配置

七、Session vs Application 模式对比

Flink on Kubernetes 支持两种部署模式，通过 `KubernetesDeploymentTarget` 枚举区分：`kubernetes-session` 和 `kubernetes-application`。两种模式在入口点、Job 图生成位置和资源隔离性上有本质差异。



图 7-1: Session 模式 vs Application 模式对比

Session 模式详解:

- 入口点: `KubernetesSessionClusterEntrypoint` -> `SessionClusterEntrypoint`
- 部署: `deploySessionCluster()` -> `deployClusterInternal()`, JM 常驻等待 Job 提交
- Job 图由客户端生成并通过 REST API 提交到已运行的 Session 集群
- 多个 Job 共享同一个 JM 和 TM 资源池, 适合交互式开发和短任务
- TM 资源按需分配, 空闲超时后自动回收

Application 模式详解:

- 入口点: `KubernetesApplicationClusterEntrypoint` -> `ApplicationClusterEntryPoint`
- 部署: `deployApplicationCluster()`, 用户 JAR 被打入镜像或通过 Artifact 上传

- JM Pod 内通过 DefaultPackagedProgramRetriever 获取 PackagedProgram 并执行 main()
- 每个 Application 独立拥有 JM 和 TM，Job 级别完全隔离
- 生产推荐模式: 更好的安全隔离和资源管理

八、K8s Service 暴露策略

Flink 通过 ServiceType 策略模式支持四种 K8s Service 类型，控制 REST API 端点的暴露方式。每种 Service 类型实现了构建外部/内部 Service 和获取端点的方法。

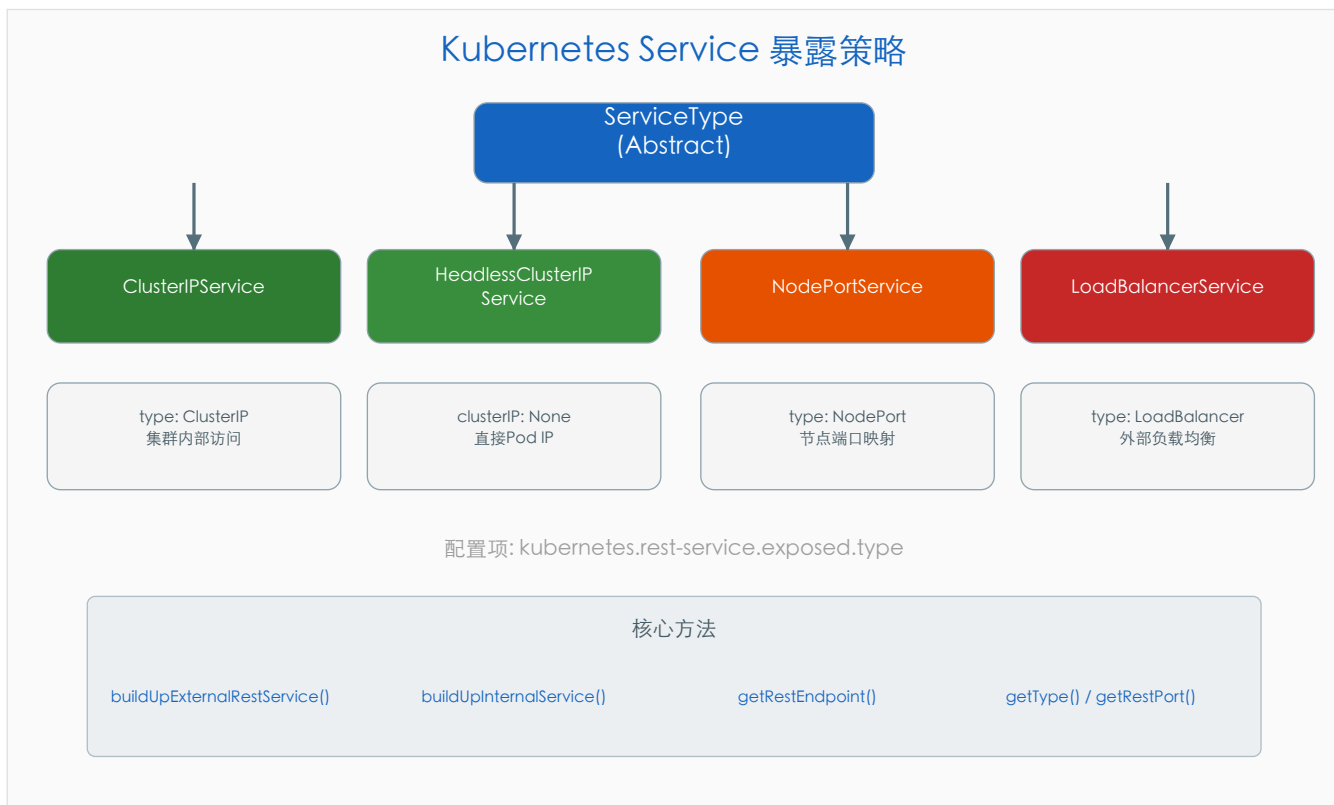


图 8-1: Kubernetes Service 暴露策略

各 Service 类型说明:

Service 类型	K8s type	端点获取方式	适用场景
ClusterIP	ClusterIP	clusterIP:port	集群内部访问(默认)
Headless ClusterIP	ClusterIP (clusterIP=None)	Pod IP 直连	HA 模式 / 直连 Pod
NodePort	NodePort	nodeIP:nodePort	开发测试环境
LoadBalancer	LoadBalancer	外部 IP:port (等待分配)	生产环境 / 云厂商

NodePort 地址类型: 通过 kubernetes.rest-service.exposed.node-port-address-type 配置，支持 InternalIP 和 ExternalIP 两种模式。

九、高可用 (HA) 机制

flink-kubernetes 模块实现了基于 K8s ConfigMap 的原生 HA 方案，替代传统的 ZooKeeper 方案。Leader 选举、状态存储和服务发现全部通过 ConfigMap 实现。

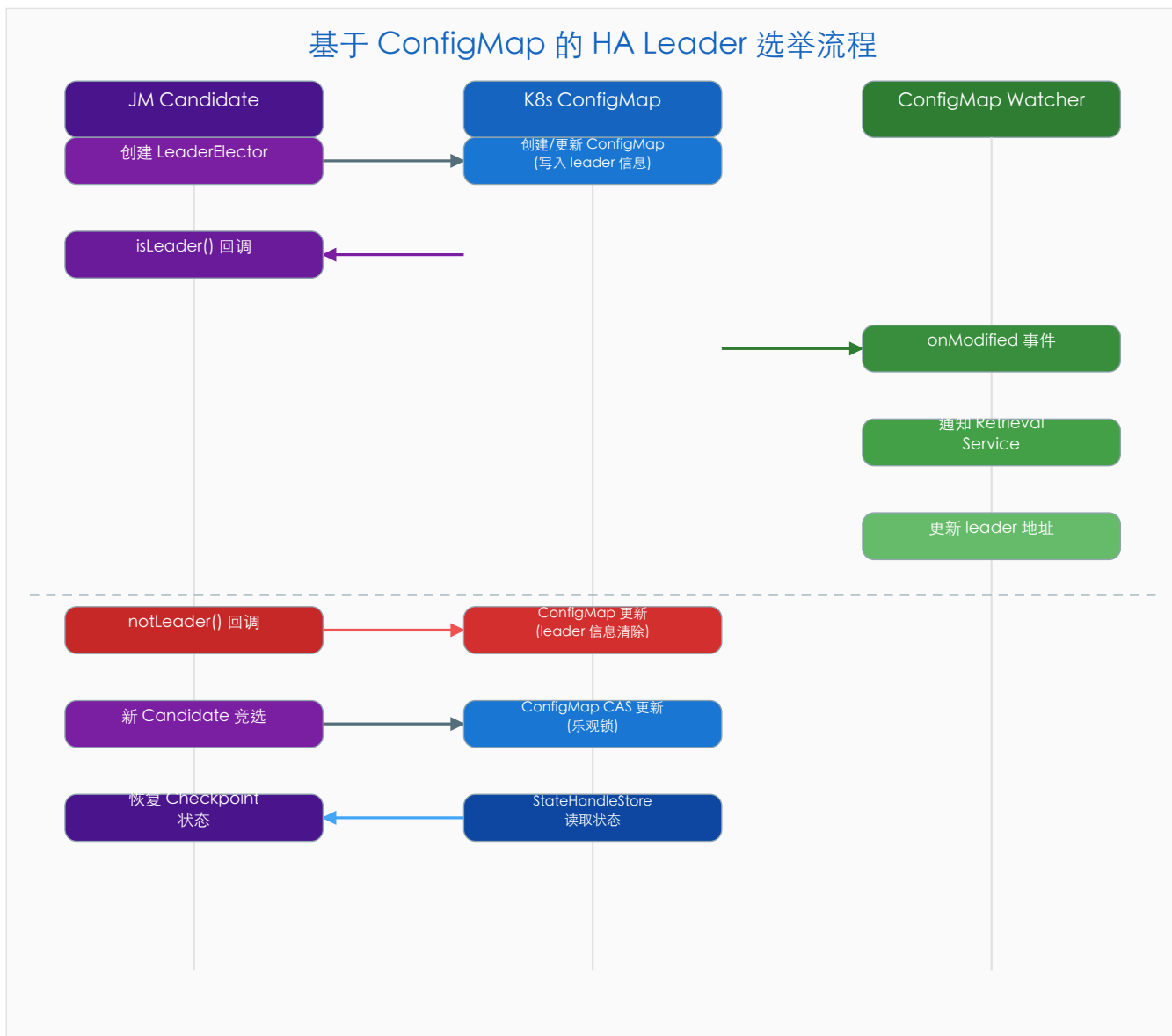


图 9-1: 基于 ConfigMap 的 HA Leader 选举流程

核心 HA 组件:

组件	职责
KubernetesLeaderElection HaServices	HA 服务实现: 继承 AbstractHaServices，创建基于 ConfigMap 的 Leader 选举和状态存储实例
KubernetesLeaderElection Driver	Leader 选举驱动: 通过 K8s LeaderElector API 实现竞选，监听 isLeader/notLeader 回调
KubernetesLeaderRetrieval Driver	Leader 发现驱动: 通过 ConfigMapSharedWatcher 监听 ConfigMap 变化，获取当前 Leader 地址

KubernetesStateHandleStore	状态存储: 基于 ConfigMap 的分布式状态存储，支持乐观锁(CAS)更新
KubernetesConfigMap SharedWatcher	共享 Watcher: 单个 K8s Watch 连接监控多个 ConfigMap，减少 API Server 负载

ConfigMap 命名规则:

- 集群 ConfigMap: {clusterId}-cluster-config-map (存储 Leader 信息)
- Job ConfigMap: {clusterId}-{jobId}-config-map (存储 Job 级别的 Checkpoint 状态)
- Leader 信息键: componentId.leader (存储 leader 的 URL 和 UUID)
- 状态句柄键: 使用 key-value 方式存储序列化的 StateHandle

乐观锁更新机制:

KubernetesStateHandleStore 和 LeaderElectionDriver 使用 checkAndUpdateConfigMap() 方法实现乐观锁更新。该方法先读取 ConfigMap 的 resourceVersion，更新后带 resourceVersion 提交，如果版本冲突(被其他节点修改)则重试。最大重试次数由 kubernetes.transactional-operation.max-retries 配置(默认5次)。

十、参数与配置体系

10.1 参数层次结构

flink-kubernetes 使用三层参数体系，从通用到特定逐层扩展：

层次	类名	核心参数
接口层	KubernetesParameters	定义 40+ 个参数获取方法 (clusterId/namespace/image/labels/tolerations/secrets/PVC 等)
抽象基类	AbstractKubernetesParameters	持有 Configuration，实现通用参数获取逻辑，231 行
JM 特化	KubernetesJobManagerParameters	CPU/Memory/LimitFactor、restPort、rpcPort、blobServerPort、serviceAccount、entrypointClass、restServiceExposedType、replicas、podTemplate、o
TM 特化	KubernetesTaskManagerParameters	podName、dynamicProperties、jvmMemOpts、containeredTMParams、externalResourceConfigKeys、blockedNodes

10.2 核心配置项 (KubernetesConfigOptions)

KubernetesConfigOptions 是最大的配置定义文件(777行，42.9KB)，包含 40+ 个配置项：

配置分类	关键配置项	说明
集群	cluster-id, namespace, context, kube-config-file	集群标识、命名空间、kubeconfig
镜像	container-image, image-pull-policy, image-pull-secrets	容器镜像、拉取策略、密钥
JM 资源	jobmanager.cpu, jobmanager.cpu-limit-factor, jobmanager.memory-limit-factor	JM CPU/内存 Request 和 Limit 比例
TM 资源	taskmanager.cpu, taskmanager.cpu-limit-factor, taskmanager.memory-limit-factor	TM CPU/内存 Request 和 Limit 比例
Service	rest-service.exposed.type	ClusterIP/NodePort/LoadBalancer/Headless
标签/注解	jobmanager-labels, taskmanager-labels, jobmanager-annotations, rest-service-annotations	Pod 和 Service 的标签/注解
Secrets	kubernetes.secrets, kubernetes.env.secret-key-ref	Secret 卷挂载和环境变量注入
Pod 模板	pod-template-file.jobmanager, pod-template-file.taskmanager	Pod 模板 YAML 文件路径
PVC	persistent-volume-claims, persistent-volume-claim-read-only	PVC 名称和只读配置
网络	hostnetwork-enabled, rest-service.exposed.node-port-address-type	HostNetwork/NodePort 地址类型
HA	jobmanager.replicas	JM 副本数(HA)
Hadoop	hadoop-conf-config-map, hadoop-conf-mount-decorator-enabled	Hadoop 配置 ConfigMap 和启用开关

Artifact	local-upload-enabled, local-upload-target, local-upload-overwrite	Artifact 上传开关/目标/覆盖策略
重试	transactional-operation.max-retries	乐观锁重试次数(默认5)
IO 线程	client-io-executor-pool-size	K8s 客户端 IO 线程池大小(默认4)

10.3 内部枚举类型

- ServiceExposedType: ClusterIP / NodePort / LoadBalancer / Headless_ClusterIP
- NodePortAddressType: InternalIP / ExternalIP
- ImagePullPolicy: IfNotPresent / Always / Never
- KubernetesDeploymentTarget: SESSION("kubernetes-session") / APPLICATION("kubernetes-application")

十一、Pod 模板支持

flink-kubernetes 支持用户自定义 Pod 模板(YAML 格式)，可对 JM 和 TM Pod 进行灵活定制。Pod 模板中的配置会被装饰器的显式配置覆盖/合并。

11.1 Pod 模板加载流程

- JM Pod 模板: 通过 `kubernetes.pod-template-file.jobmanager` 配置文件路径
- TM Pod 模板: 通过 `kubernetes.pod-template-file.taskmanager` 配置文件路径
- JM 端加载: `KubernetesClusterDescriptor.deployClusterInternal()` 中通过 `loadPodFromTemplateFile()` 加载，传给工厂
- TM 端传递: `PodTemplateMountDecorator` 将 TM Pod 模板内容写入 `ConfigMap`，挂载到 JM Pod
- TM 端加载: `KubernetesResourceManagerDriver.initializeInternal()` 从挂载文件加载 TM Pod 模板

11.2 模板合并规则

- 模板中主容器名必须为 "flink-main-container"，否则使用空容器初始化
- `FlinkPod` 将 Pod 拆分为 `podWithoutMainContainer` + `mainContainer`，分别处理
- 装饰器的显式配置优先于模板配置(装饰器会覆盖或追加)
- 模板中的自定义 sidecar 容器、init containers 等会被保留
- 资源限制(CPU/Memory)由装饰器设置，模板中的设置会被覆盖

11.3 Pod 模板示例

一个典型的 TM Pod 模板 YAML:

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    custom-annotation: custom-value
spec:
  containers:
    - name: flink-main-container # ■■■■■■■■
  volumeMounts:
    - name: custom-volume
      mountPath: /opt/custom
    - name: sidecar # ■■■■ sidecar ■■
  image: my-sidecar:latest
  volumes:
    - name: custom-volume
      emptyDir: {}
```


十二、关键设计模式与架构思想

装饰器模式 (Decorator)

KubernetesStepDecorator 链式增强

FlinkPod，每个装饰器关注单一职责(镜像设置、命令注入、配置挂载、Service 创建等)。这是模块最核心的设计模式，实现了 Pod 配置的高度可扩展性和可组合性。

工厂模式 (Factory)

FlinkKubeClientFactory(单例)、KubernetesClusterClientFactory(SPI 注册)、KubernetesResourceManagerFactory(单例)、KubernetesJobManagerFactory/KubernetesTaskManagerFactory(静态工厂) — 所有核心组件通过工厂创建，解耦实例化逻辑。

策略模式 (Strategy)

ServiceType 抽象不同的 K8s Service 暴露策略(ClusterIP/HeadlessClusterIP/NodePort/LoadBalancer)，运行时根据配置选择具体策略。

建造者模式 (Builder)

FlinkPod.Builder 链式构建 Pod 对象；Fabric8 的

PodBuilder/ContainerBuilder/DeploymentBuilder 用于构建 K8s 资源对象。

观察者模式 (Observer)

FlinkKubeClient.WatchCallbackHandler<T> 处理 Pod/ConfigMap Watch 事件(onAdded/onModified/onDeleted/onError)，实现 K8s 资源的实时监控。

模板方法模式 (Template Method)

AbstractResourceManagerDriver.initializeInternal() 由子类

KubernetesResourceManagerDriver 实现；AbstractKubernetesParameters 定义参数获取骨架，子类实现特定参数。

单例模式 (Singleton)

FlinkKubeClientFactory.INSTANCE / KubernetesResourceManagerFactory.INSTANCE — 确保全局唯一的客户端工厂和资源管理工厂。

SPI 服务发现

KubernetesSessionClusterExecutorFactory 通过 META-INF/services

注册，实现插件化部署目标。KubernetesClusterClientFactory 同样通过 SPI 机制自动发现。

CompletableFuture 异步编排

requestResource() 返回 `CompletableFuture<KubernetesWorkerNode>`，Pod 创建后 `complete()`。Fabric8FlinkKubeClient 内部使用 `ScheduledExecutorService` 实现异步 IO。

OwnerReference 级联删除

所有附属资源(Service/ConfigMap/TM Pod)的 OwnerReference 设为 JM Deployment，删除 Deployment 时 K8s 自动级联删除全部关联资源，避免资源泄漏。